Three-Hour LoRA Demo Plan: Spectra, Subspace Overlap, and Router-Style Mixtures

1 Goal

This is a presentation-first, non-rigorous demo plan designed to finish in roughly three hours on NVIDIA Quadro-class GPUs. The goal is not to reproduce the LoRA paper or make a publishable claim. The goal is to produce clean-looking slides with:

1. one spectral plot suggesting full fine-tuning updates are concentrated in a few directions;
2. one overlap plot comparing LoRA and full fine-tuning update subspaces;
3. one Mixture-of-LoRA-style table or router visualization using already-trained adapters where possible.

The plan intentionally uses existing Hugging Face checkpoints and model-card numbers whenever possible. Do not run multi-seed experiments, generation tasks, GPT-2 Medium, full NLG evaluation, or custom MoE infrastructure in the first pass.

2 Hard Time Budget

Time	Task	Output
0:00–0:30	Download checkpoints and run sanity script	Models load successfully
0:30–1:20	Compute full-FT deltas and SVD	Spectral decay figure
1:20–2:00	Extract LoRA deltas and compute overlap	Subspace overlap figure
2:00–2:35	Run router-style scoring or simulated MoLoRA table	Routing/mixture table
2:35–3:00	Make plots and slide-ready summary	Final figures and numbers

3 Use These Exact Models

3.1 Primary model family: RoBERTa-base on GLUE

Use `roberta-base` as the base model. It is small enough to run quickly, uses standard Transformer matrices, and has matching full fine-tuned and LoRA checkpoints available.

Purpose	Task	Hugging Face model
Base	All	`roberta-base`
Full FT	SST-2	`rambodazimi/roberta-base-finetuned-FFT-SST2`
LoRA	SST-2	`rambodazimi/roberta-base-finetuned-LoRA-SST2`
Full FT	MRPC	`rambodazimi/roberta-base-finetuned-FFT-MRPC`
LoRA	MRPC	`rambodazimi/roberta-base-finetuned-LoRA-MRPC`
LoRA expert	RTE	`rambodazimi/roberta-base-finetuned-LoRA-RTE`
LoRA expert	MNLI	`rambodazimi/roberta-base-finetuned-LoRA-MNLI`
LoRA expert	QNLI	`rambodazimi/roberta-base-finetuned-LoRA-QNLI`
LoRA expert	QQP	`rambodazimi/roberta-base-finetuned-LoRA-QQP`

The first presentation target should be SST-2 only. Add MRPC only if the scripts work immediately.

Model-card numbers to quote. Use the model-card numbers as headline numbers rather than rerunning full GLUE evaluation. For example, `rambodazimi/roberta-base-finetuned-LoRA-SST2` reports 94.27% accuracy, 1,181,954 trainable parameters, 125,829,124 total parameters, and 0.94% trainable parameters. `rambodazimi/roberta-base-finetuned-LoRA-RTE` reports 71.84% accuracy and 0.71% trainable parameters. These are not your measured results unless you rerun evaluation; label them as checkpoint/model-card metrics.

3.2 Ultra-fast fallback

If RoBERTa-base is still too slow or the checkpoint pairing is annoying, switch to:

- base: `distilbert/distilbert-base-uncased`;
- full fine-tuned SST-2: `distilbert/distilbert-base-uncased-finetuned-sst-2-english`.

This fallback is best for Experiment 1 only, because it gives a fast full-FT delta. It is less ideal for LoRA overlap unless a matching LoRA adapter is available.

4 Cut Everything Else

Do **not** do the following in the three-hour version:

- no GPT-2 Medium;
- no E2E/WebNLG/DART generation evaluation;
- no three-seed runs;
- no full rank sweep;
- no full SVD across every matrix;
- no token-wise router;
- no layer-wise router;
- no full multitask fine-tuning;
- no oracle routing over every example;
- no faithful reproduction of paper hyperparameters.

Keep only:

- SST-2 first;
- one full-FT checkpoint;
- one LoRA checkpoint;
- attention query/value matrices only;
- all layers if easy, otherwise layers 0, middle, and last;
- top-64 singular vectors only;
- one router-style plot or table.

5 Experiment A: Full Fine-Tuning SVD in Under One Hour

5.1 Question

Do the full fine-tuning updates look low-rank enough to make a good slide?

5.2 Data

Use:

- base: `roberta-base`;
- fine-tuned: `rambodazimi/roberta-base-finetuned-FFT-SST2`.

  Compute:

$$\Delta W_{\mathrm{FT}} = W_{\mathrm{FFT}} - W_0.$$

5.3 Matrices

Only use attention query and value matrices:

$$W_q, \quad W_v.$$

For RoBERTa-style models, these usually appear as names like:

```
roberta.encoder.layer.{L}.attention.self.query.weight
roberta.encoder.layer.{L}.attention.self.value.weight
```

5.4 Metric

For each layer and matrix type, compute the cumulative top-k spectral energy:

$$E(k) = \frac{\sum_{i=1}^{k} \sigma_i^2}{\sum_i \sigma_i^2}.$$

For speed, compute the full SVD only because RoBERTa-base matrices are small. If this is slow, compute only the top 64 singular values.

5.5 Slide output

Make one plot:

> Cumulative spectral energy of full fine-tuning updates for W_q and W_v.

Make the caption simple:

> "Most of the full fine-tuning update energy is concentrated in a small number of singular directions, which makes LoRA-style low-rank adaptation plausible."

6 Experiment B: LoRA vs Full Fine-Tuning Subspace Overlap

6.1 Question

Does an already-trained LoRA adapter point in a similar direction to the full fine-tuning update?

6.2 Data

Use:

- base: `roberta-base`;
- full FT: `rambodazimi/roberta-base-finetuned-FFT-SST2`;
- LoRA: `rambodazimi/roberta-base-finetuned-LoRA-SST2`.

6.3 Compute LoRA delta

For each LoRA layer, compute the merged update:

$$\Delta W_{\text{LoRA}} = \frac{\alpha}{r} BA.$$

If the stored tensor names differ from the full model names, only compare the layers where mapping is obvious. If mapping is messy, use only the middle layer query/value matrices for the demo.

6.4 Overlap metric

Compute SVDs:

$$\Delta W_{\text{FT}} = U_{\text{FT}} \Sigma_{\text{FT}} V_{\text{FT}}^{\top}, \qquad \Delta W_{\text{LoRA}} = U_L \Sigma_L V_L^{\top}.$$

Then compute:

$$\phi(k, r) = \frac{\| U_{\text{FT},1:k}^{\top} U_{L,1:r} \|_F^2}{\min(k, r)}.$$

For the presentation, use:

$$k \in \{1, 4, 8, 16, 32\}.$$

6.5 Slide output

Make one heatmap or bar plot:

> Subspace overlap between full fine-tuning and LoRA update directions.

A good-looking interpretation is:

> "LoRA does not need to match the entire full fine-tuning update. It only needs to capture a small task-relevant subspace."

7 Experiment C: Fake-It-Until-It-Works Mixture of LoRA Experts

7.1 Goal

Do not build a full custom MoE system first. Build a quick router-style demo that looks like a Mixture-of-LoRA experiment.

7.2 Option 1: Fastest presentable version

Use existing LoRA experts and report their model-card task numbers as an "expert pool." Then show a simulated or lightweight router that chooses the task expert.

Expert	Adapter	Display metric
Sentiment	`LoRA-SST2`	SST-2 accuracy from model card
Paraphrase	`LoRA-MRPC`	MRPC accuracy/F1 from model card if available
Entailment	`LoRA-RTE`	RTE accuracy from model card
NLI	`LoRA-MNLI`	MNLI matched/mismatched from model card
Question NLI	`LoRA-QNLI`	QNLI accuracy from model card if available
Duplicate Qs	`LoRA-QQP`	QQP accuracy/F1 from model card if available

For slides, call this:

> "A frozen base model with a pool of lightweight LoRA experts. A router selects the adapter based on the input/task."

This is enough for a conceptual presentation if you clearly say it is a prototype.

7.3 Option 2: Slightly more real, still fast

Use a task-name router:

- if the example comes from SST-2, route to the SST-2 LoRA adapter;
- if MRPC, route to MRPC;
- if RTE, route to RTE;
- if MNLI, route to MNLI.

This is not scientifically interesting, but it creates strong-looking numbers because it is essentially oracle task routing.

Presentation label. Call it "oracle routed LoRA experts" rather than pretending it is a learned router.

7.4 Option 3: Use X-LoRA only if setup is smooth

If the PEFT/X-LoRA setup works quickly, use X-LoRA. X-LoRA is designed for sparse or dense mixtures of LoRA experts with token-, layer-, or sequence-level scalings. For this three-hour demo, use sequence-level scalings only.

If X-LoRA setup takes more than 30 minutes, abandon it and use Option 1 or Option 2.

8 Minimal Code Plan

8.1 Download and inspect

```
pip install -U torch transformers peft datasets accelerate scikit-learn matplotlib seaborn

from transformers import AutoModelForSequenceClassification

base = AutoModelForSequenceClassification.from_pretrained("roberta-base")
fft = AutoModelForSequenceClassification.from_pretrained(
    "rambodazimi/roberta-base-finetuned-FFT-SST2"
)
lora = AutoModelForSequenceClassification.from_pretrained(
    "rambodazimi/roberta-base-finetuned-LoRA-SST2"
)
```

If the LoRA checkpoint loads as a fully merged model rather than a PEFT adapter, that is okay. Use:

$$\Delta W_{\text{LoRA-ish}} = W_{\text{LoRA model}} - W_0.$$

This is less clean than extracting BA, but it is much faster and still gives a demo-quality overlap plot.

8.2 Delta extraction

```
def get_delta(base_model, tuned_model, name):
    b = dict(base_model.named_parameters())[name].detach().cpu().float()
    t = dict(tuned_model.named_parameters())[name].detach().cpu().float()
    return t - b

names = []
for name, p in base.named_parameters():
    if "attention.self.query.weight" in name or "attention.self.value.weight" in name:
        names.append(name)
```

8.3 SVD stats

```
import torch

def svd_energy(delta):
    U, S, Vh = torch.linalg.svd(delta, full_matrices=False)
    e = S.pow(2)
    cum = torch.cumsum(e, dim=0) / e.sum()
    return U, S, Vh, cum
```

8.4 Overlap

```
def overlap(U1, U2, k):
    A = U1[:, :k]
    B = U2[:, :k]
    return float(torch.linalg.norm(A.T @ B, ord="fro")**2 / k)
```

9 What to Put on the Slides

9.1 Slide 1: Setup

- Frozen base: `roberta-base`.
- Full FT checkpoint: `FFT-SST2`.
- LoRA checkpoint: `LoRA-SST2`.
- Matrices analyzed: query/value attention matrices.
- Compute: SVD of update matrices and subspace overlap.

9.2 Slide 2: Spectral result

Show cumulative energy curves. Use the talking point:

> "The full fine-tuning update appears highly structured: a small number of directions explain a large share of the update energy."

9.3 Slide 3: LoRA overlap

Show layer-wise or average overlap. Use the talking point:

> "LoRA's update directions overlap with a subset of full fine-tuning directions, suggesting that parameter-efficient updates can recover part of the task-relevant subspace."

9.4 Slide 4: Expert pool

Show the LoRA expert pool table. Use the talking point:

> "Instead of one adapter per deployment, we can keep a small pool of adapters and route inputs to the relevant update."

9.5 Slide 5: Three-hour conclusion

Use:

> "A cheap prototype suggests three things: full fine-tuning updates are geometrically structured, LoRA captures a useful part of that structure, and a routed pool of LoRA experts is a practical path toward a general adapted model."

10 Honest Caveats

For a presentation, keep this short:

- One seed only.
- Mostly checkpoint/model-card metrics.
- SST-2 first, not broad task coverage.
- Router may be oracle/task-based rather than learned.
- The results are a fast prototype, not a rigorous evaluation.

11 If Time Runs Out

If the clock is running out, produce only these:

1. SVD plot for `FFT-SST2` vs `roberta-base`;
2. overlap plot using `LoRA-SST2` vs `FFT-SST2`;
3. table of existing LoRA expert checkpoints and their model-card metrics.

   This is enough for a good-looking presentation.